

AtomicPass

Concurrency-safe, rush-proof event ticketing

A college fest releases **5,000 passes at 6 PM**. About 5,000 students press **Buy** in the same second.

- 1 **Never oversell.** Not by one ticket, not across tiers, not under concurrency.
- 2 **Free the seats** of checkouts nobody pays for, so others can buy them.
- 3 **Admit each pass exactly once** at the gate.

We read how **Hi.Events** (open source, PHP/Laravel) handles this, listed where it is weak, and rebuilt the core in **TypeScript** so that each rule is **one atomic database step** that a test tries to race.

One process

One SQLite file

No external service

No API key

No AI in the product

Two improvements on top of the baseline:

Fix: capacity enforced on tier *and* event pool, counting holds (GAP-1).

Differentiator: a waiting room and a per-buyer cap.

How Hi.Events handles a rush, and where it is weak

Commit `af22b01` · static read of the source, nothing executed · evidence from docs/OBSERVATIONS.md

WHAT IT DOES	EVIDENCE (FILE:LINE)	WEAK FOR A 6 PM RUSH
One Postgres advisory lock per event wraps all order creation	CreateOrderHandler.php:61-62 TransactionLockService.php:21-27	All 5,000 creates queue behind one lock; no queue in front (GAP-2)
A hold is an order marked RESERVED; an expired hold is only filtered out when counting	OrderItemRepository.php:81-88 no expiry job: Console/Kernel.php:22-34	Rows stay RESERVED; nothing tells the waitlist a seat is free (GAP-5)
The locked check for a shared pool subtracts sales but not live holds	AvailableProductQuantitiesFetchService.php:75-83 CapacityAssignmentDomainObject.php:78-85	A pool shared by tiers can be oversold (GAP-1, from the code, not reproduced)
“Sold” is counted later, by an unguarded +N update when payment completes	ProductQuantityUpdateService.php:38-42, 99-106	Hold to sold is not one atomic step (GAP-8)
Admin check-in reads, then writes; the scanner path has a unique index	CheckInAttendeeHandler.php:78-123 ...add_unique_attendee_check_in_index.php:22-26	Two paths, two sources of truth; one can admit twice (GAP-6)
An exhausted promo is dropped silently; limits are per order and per IP only	CreateOrderHandler.php:116-118 OrderCreateRequestValidationService.php:489 RouteServiceProvider.php:57-60	Full-price surprise (GAP-9); hoarding and campus NAT (GAP-3, GAP-4)

Several findings (GAP-1, 2, 3, 4, 8) are reasoned from the cited lines and were not reproduced; the documents say which. Full list: docs/OBSERVATIONS.md and docs/GAPS.md.

Every rule is one conditional update that a test races

A `held` and a `sold` counter on every tier **and** on the event pool. The number of rows an UPDATE changes decides who won.

DIFFERENTIATOR

1 · Waiting room

First come, first admitted; a few per tick. An admission is used up only by a hold that succeeds. **Per-buyer cap** on the normalised email.

THE FIX

2 · Hold

One `BEGIN IMMEDIATE` : tier, then pool.

```
UPDATE tiers SET
held=held+q
WHERE cap-sold-held >=
q;
UPDATE events SET
held=held+n
WHERE cap-sold-held >=
n;
```

0 rows changed: refused, all rolled back.

3 · Pay

`held` → `sold` in the same statements, so held + sold never changes. Creates the order and one ticket per seat with a 128-bit QR token.

4 · Expire

Every reserve, pay and read frees due holds first; a sweeper only tidies up. **A seat never waits for the sweeper.**

5 · Gate

```
UPDATE tickets
SET status='CHECKED_IN'
WHERE qr_token=? AND
status='VALID';
```

One scan wins. Refund uses the same guard, so a scan and a refund racing have one winner.

Database as the last line: `CHECK (sold + held ≤ capacity)` on every tier and event.

An audit after every test: counters are recomputed from the rows; any difference fails the test.

Honest ceiling: SQLite has one writer, so one machine. The same statements are documented for PostgreSQL, not run.

EVIDENCE

Run, repeated, audited

Real output from this repository on the development laptop (Windows 11, Node 24.13).

```
$ npm run killer -- --reporter=verbose
✓ kt1          two buyers ask for the last ticket at once: exactly one 201, th...
✓ kt3          the first scan admits, the second is 409 ALREADY_CHECKED_IN wit...
✓ kt1-pool     tiers A and B (5 each) in a pool of 5: ten parallel holds, exac...
✓ buyer-cap-race ten parallel requests of one seat from one buyer: exactly 4 suc...
✓ kt1-pool     a hold that spans both tiers is taken whole or not at all
✓ kt3          twenty parallel scans of a new ticket: exactly one 200, ninetee...
✓ kt3          each ticket of an order admits once, independently
✓ kt3-db       two worker threads scan the same QR at the same moment: exactly...
✓ queue-race   two connections use the same admission at the same moment: exac...
✓ kt1-db       two worker threads race for the last ticket: exactly one wins, ...
✓ buyer-cap-race two connections ask for 3 seats each for the same buyer at the ...
✓ refund-scan-race exactly one wins for every ticket, and the counters stay true
✓ kt1          exactly 10 successes every run, and sold + held = 10
✓ kt2          buyer 2 is refused, waits out the TTL, then gets the seat; payi...
```

```
Test Files  9 passed (9)
Tests      14 passed (14)
```

20 / 20

runs of `npm run killer` in a row on the final code, all green

163 tests

in 24 files; after each one an audit recomputes every counter from the rows and fails the test on any difference

4,500 + 500

`npm run rush`: 5,000 buyers, 4,500 seats: 4,500 holds, 500 sold out, nothing oversold, invariants clean (4.9 s, one run)

The tests can fail. Each guard was removed once, the tests run, the edit reverted. A deferred transaction instead of `BEGIN IMMEDIATE` made KT1-db and the per-buyer cap race fail ("database is locked"); dropping the pool guard failed KT1-pool; dropping the `VALID` guard failed every KT3 test, over HTTP and across two connections.

Five commands, then the live demo

```
# Node.js 22.12 or newer
git clone https://github.com/Yash13606/debugdevise
cd debugdevise
npm install
npm run killer
npm start
```

Short hold lifetime for the demo. Put this in `.env` (copy of `.env.example`):

```
HOLD_TTL_SECONDS=2
```

Every setting has a default, so without `.env` a hold lasts 600 s.

1. **Show** `.env` with `HOLD_TTL_SECONDS=2`.
2. **`npm run killer`**
last-ticket race, pool race, an unpaid hold frees its seat, one scan per ticket, and the races across connections
3. **`npm run demo`**
ten buyers join the line, three admitted per tick, an unadmitted buyer gets `403 NOT_ADMITTED`, the cap refuses extra tickets with `409 BUYER_LIMIT`
4. **Open the matching section of `docs/`** for each point.
5. **Optional: `npm run rush`**
5,000 buyers press buy at once for 4,500 seats